

PLIEGO — Dossier completo del producto y del código

Documento de referencia para defensa TFM, onboarding y prompts a agentes.

Fuente de verdad: el repositorio y el despliegue actuales (no marketing inventado).

Fecha de este dossier: 2026-10-01 · commit de referencia en `main` tras PDF Chromium + plantillas 3 actos.

0. Cómo usar este documento

1. **Como brief humano:** léelo de arriba abajo antes de la defensa.
 2. **Como prompt a un agente:** copia la sección *Prompt maestro* al final + pega las dudas concretas.
 3. **Como PDF:** ábrelo en VS Code / Obsidian / GitHub y "Print → Save as PDF", o pásalo por Pandoc.
-

1. Qué es PLIEGO (una frase)

PLIEGO es un estudio editorial visual en el navegador: creas revistas, portadas, lookbooks y presentaciones con un canvas real (capas, tipografía, motion, CTAs), autoguardas, versionas, exportas **PDF interactivo** y publicas un enlace `/p/:slug`.

Posicionamiento documentado: *entre InDesign (control tipográfico/editorial) y Canva (facilidad)*, con estética de estudio creativo (oscuro, neón, tipografía display), no de panel genérico.

- Tagline de producto: *estudio editorial visual*
- Claim de landing: *"Diseña revistas, portadas y sistemas visuales con la fluidez de un estudio creativo."*
- Demo viva: *"No es un panel. No es otra herramienta. Es una portada que late."*

2. Finalidad y valor

Finalidad

Permitir que un diseñador / editor / estudiante produzca **piezas editoriales digitales publicables** (no solo mockups estáticos) sin instalar InDesign ni quedarse en plantillas rígidas tipo Canva.

Productos / entregables que aporta

Entregable	Qué es
Proyecto en editor	Lienzo multi-página con elementos tipográficos, formas, imagen, botón, vídeo
Plantillas PLIEGO	6 plantillas con 3 actos (portada / cuerpo / cierre)
Versiones	Snapshots manuales restaurables del JSON del documento
PDF interactivo	Descarga con enlaces GoTo entre páginas y URI externas
Publicación web	URL pública <code>/p/:slug</code> con vista zoomable
Demo viva	Experiencia de marketing/producto en <code>/demo</code>
Identidad de marca embebida	Tokens, tipografías, bloques editoriales, swatches

Quién lo usaría

- Estudiantes de diseño / comunicación visual
- Estudios pequeños que publican lookbooks o decks
- Marcas culturales / editoriales que necesitan un link vivo, no un PDF plano
- El propio autor del TFM como portfolio demostrable

3. Qué hace y qué no hace

Sí hace (implementado)

1. Registro / login / logout / sesión por cookie JWT

2. CRUD de proyectos (crear, listar, duplicar, archivar, eliminar)
3. Plantillas + lienzo en blanco (story, square, landscape, presentation)
4. Editor Konva: páginas, capas, propiedades, zoom, transform, undo/redo (50), copy/paste
5. Texto, formas (rect/ellipse + gradientes), imagen, botón (CTA), vídeo (placeholder + link)
6. Animaciones preset (fade, scale, slide, blur...) al load/scroll en vista pública
7. Interacciones `pliego:next|prev|page:...`, URLs, mailto, paths
8. Autoguardado debounce ~800 ms + indicador idle/saving/saved/error
9. Versiones manuales + restore
10. Subida de imágenes (JPEG/PNG/WebP/GIF ≤ 5 MB)
11. Exportación PDF en servidor (Puppeteer + pdf-lib)
12. Publicar / despublicar por slug
13. Demo viva + landing con identidad PLIEGO
14. Tests unitarios + CI GitHub Actions
15. Deploy Render (Node) + Docker opcional

No hace (fuera del slice / no existe en código)

- Colaboración en tiempo real / multiplayer
- Pagos, planes, Stripe, billing
- Object storage S3 (assets en disco local del servidor)
- Conflictos multi-pestaña (último guardado gana)
- Cola de jobs para PDFs / límites duros de páginas
- Migración automática de esquemas JSON antiguos
- Admin bypass real pese a campo `role`
- Suite E2E Playwright amplia
- Auditoría WCAG formal
- PostgreSQL en el deploy free actual (sigue SQLite efímera en Render)

4. Cómo está construida

Monorepo

Pliego_TFM_KilianTorres/

- └─ frontend/ React 19 + Vite + Zustand + Tailwind + Konva + Framer Motion
- └─ backend/ Express 5 + Prisma + Zod + JWT + Puppeteer + pdf-lib
- └─ docs/ Arquitectura, DB, roadmap, este dossier
- └─ docker/ Compose / Nginx
- └─ render.yaml Blueprint Render free (runtime Node)
- └─ scripts/ start-web (sirve API + estáticos del build)

Flujo de datos

```
UI (React) → Zustand editorStore → API (cookie JWT)
                                     → Prisma (User, Project.documentJson, Versions, Assets, Exports)
                                     → Disco (uploads/, exports/)
Export PDF: documentJson → HTML escapado → Chromium headless → pdf-lib anota links → fichero PDF
Public: GET /api/public/:slug → PublicRenderer / ZoomablePublicView
```

Modelo de documento (contrato)

```
DocumentModel {
  version: 1
  pages: Page[]
  meta: { title, width, height, templateId?, templateName? }
}
Page { id, name, order, background, elements[] }
CanvasElement = Text | Image | Shape | Button | Video
// + effects, animation, interaction
```

Persistencia viva: una columna `projects.documentJson` (string JSON).

Versiónes: tabla `ProjectVersion` con snapshot completo.

Stack y porqués cortos

Pieza	Elección
Canvas	Konva + react-konva (declarativo, capas, encaja con Zustand)
Estado	Zustand (ligero, JSON-first, undo stack simple)
API	Express + capas routes/controllers/services/repositories
Validación	Zod
Auth	JWT en cookie httpOnly <code>pliego_token</code>
DB demo	SQLite + WAL
PDF	puppeteer-core + system Chrome o <code>@sparticuz/chromium</code> en Render
UI	Tailwind + tokens <code>PLIEGO_BRAND</code> + tipografías Syne / Space Grotesk / DM Sans / Instrument Serif / JetBrains Mono

Plantillas (todas 3 páginas)

ID	Formato
manifiesto-digital	1440×900 landscape
portfolio-kinetic	1080×1350 portrait
portada-editorial	1080×1350
revista-doble	1200×1600
catalogo-producto	1080×1350
presentacion-slide	1920×1080

Acciones de usuario (mapa)

Landing: Empezar / Login / Registro / Ver demo

Dashboard: Nueva desde plantilla / blank / abrir / duplicar / archivar / eliminar / publicar

Editor: añadir texto/forma/imagen/botón/vídeo · capas · páginas · props · brand swatches · bloques editoriales · undo/redo · zoom (pinch/dock en móvil) · guardar versión · exportar PDF · publicar

Público: navegar páginas, CTAs, motion al scroll, zoom

Estética visual

- Fondo ink #050608 / #0B0E11 , acentos neón #4F80FF , rosa #FF4EDB , violeta, lima #B2FF3A
- Display **Syne**, UI **Space Grotesk**, body **DM Sans**, metadatos **JetBrains Mono**, voz editorial **Instrument Serif**
- Mesh/gradientes brand-flow, posters de plantilla estáticos (sin flicker), reduce-motion respetado
- Sensación: estudio Figma/Linear + portada de revista digital, no dashboard SaaS genérico

Código — datos que suelen faltar en la defensa

- ~9.3k líneas TS/TSX de aplicación (front+back, sin node_modules)
- Ownership siempre vía `projectService.getOwned(projectId, ownerId)`
- PDF: `escapeHtml` en textos/labels/URLs embebidas; hotspots `URI` / `GoTo`
- Autosave: `markDirtyAndScheduleSave` → 800 ms → `PATCH .../document`
- History local: `HISTORY_LIMIT = 50`
- Slug: `slugify(title) + '-' + 6 chars base36`
- Uploads: whitelist MIME + `MAX_UPLOAD_BYTES=5242880`
- Render free: SQLite efímera, cold start ~1 min, Chromium vía Sparticuz
- Tests: **11** (9 backend + 2 frontend), todos pasando en local

5. Despliegue y acceso

Entorno	URL
Local front	http://127.0.0.1:45321
Local API	http://127.0.0.1:45322/api/health
Producción (Render)	https://pliego-kr1j.onrender.com

Health prod verificado: `{"ok":true,"service":"pliego-api","env":"production","db":"up"}` .

Cuenta demo (seed): `demo@pliego.app` / `demo1234`

Repo: https://github.com/kiliente2000-source/Pliego_TFM_KilianTorres

Nota free tier: tras ~15 min idle se duerme; el primer hit puede tardar. Redeploy puede resetear SQLite.

6. Lo positivo del proyecto (argumentario TFM)

1. **Ciclo completo real:** diseñar → versionar → PDF interactivo → publicar link.
2. **Modelo de documento propio** (no atado a Fabric/Konva serialization).
3. **Identidad de producto coherente** (código + UI + plantillas + demo).
4. **Decisiones técnicas defendibles** (tabla de preguntas abajo).
5. **Deploy público demostrable**, no solo localhost.
6. **Seguridad básica bien planteada** (httpOnly, ownership, MIME, escape HTML, rate limit).
7. **Mobile editor** pensado (dock, zoom, fit one-shot).
8. **PDF no es “imprimir pantalla”:** anota enlaces para Acrobat/Preview.

Límites honestos (también es positivo reconocerlos): tests aún pocos, sin collab, SQLite en free, sin cola PDF, a11y parcial.

7. Preguntas de defensa — respuestas claras

¿Por qué SQLite y no PostgreSQL o MySQL?

Para el TFM/demo: cero ops, un fichero, Prisma idéntico, WAL para lecturas concurrentes ligeras. El schema y `docs/DATABASE.md` están preparados para cambiar `provider` a `postgresql` o `mysql` sin rediseñar entidades. En Render free, SQLite es lo pragmático (aunque **efímera** entre redloys). En producción sería: PostgreSQL gestionado.

¿Por qué guardas el documento como JSON y no en tablas (páginas, elementos)?

Porque el documento es un **grafo editorial flexible** (tipos heterogéneos, effects, animation, interaction, zIndex). Normalizarlo a filas explotaría en JOINS y migraciones

por cada prop nueva. El JSON es el contrato del editor, del undo, de las versiones y del PDF. Las tablas relacionales guardan lo estable: User, Project, Version, Asset, Export.

Si mañana cambias el formato del JSON, ¿qué pasa con los documentos antiguos?

Hoy el contrato exige `version: 1 ( isDocumentModel )`. **No hay migrador automático aún.** Estrategia correcta a documentar:

1. Subir `meta.version / document.version` a 2.
2. Función `migrateDocument(doc)` en load (API + frontend).
3. Reescribir al guardar.

Sin eso, un cambio breaking devolvería “documento inválido”. Las versiones antiguas en `ProjectVersion` seguirían en v1 hasta migrarlas.

¿Por qué Konva y no Fabric.js o canvas a mano?

Documentado en `docs/ARCHITECTURE.md`: Konva + react-konva es **declarativo en React**, capas nativas, buen TypeScript, y el estado vive en Zustand (inmutable) en lugar del modelo mutable de Fabric. Canvas a mano implicaría reimplementar hit-testing, transformadores, capas y DPI. Konva renderiza; el dominio es nuestro JSON.

¿Por qué exportas en el servidor con Puppeteer y no en el cliente?

- Mismas fuentes/medidas en un Chromium controlado.
- No dependes del navegador del usuario ni de popups/print dialog.
- Puedes anotar el PDF con pdf-lib (GoTo/URI) de forma fiable.
- En cliente: peso enorme, inconsistencia tipográfica, y peor control de seguridad de assets.

En Render sin apt: `@sparticuz/chromium` resuelve el binario.

¿El PDF sale igual que el lienzo?

Muy fiel, no pixel-perfect. El pipeline re-renderiza el documento a HTML/CSS absolutos con las mismas medidas `width×height`, tipografías de Google Fonts y gradientes. Diferencias posibles: antialiasing Konva vs Chromium, vídeos (marcador), algunos blend modes/effects, fuentes si fallan a cargar (hay espera corta post- `load`). Los CTAs sí se refuerzan como anotaciones PDF.

¿Qué pasa si dos personas exportan a la vez, o si el PDF tiene 50 páginas?

- **No hay cola:** cada export lanza un Chromium. Dos a la vez = dos procesos (en free tier: riesgo de OOM / timeout).
- Ownership evita exportar proyectos ajenos, pero no hay rate limit específico de export.
- 50 páginas: posible en teoría (HTML multipágina + `page-break`), pero memoria/tiempo crecen linealmente; no hay límite explícito ni progreso streaming. Mejora futura: cola Bull/Redis + límite de páginas + Chromium compartido.

¿Cada cuánto se guarda? ¿Y si tengo el mismo proyecto abierto en dos pestañas?

Debounce ~**800 ms** tras cada cambio. Indicador saving/saved/error.

Dos pestañas: no hay sync ni locking; **el último PATCH gana** y puede pisar la otra.

Undo es solo local a la pestaña. Mitigación futura: `updatedAt` / ETag /

BroadcastChannel.

¿Por qué Zustand y no el estado de React o Redux?

El editor necesita un store **fuera del árbol**, con mutaciones frecuentes, historial past/future y serialización JSON. React Context re-renderiza de más; Redux es más ceremonia para el mismo patrón. Zustand es mínimo, tipado, y encaja con “el canvas solo pinta el store”.

¿Cómo evitas que un usuario vea o modifique los proyectos de otro?

1. Cookie JWT verificada (`requireAuth`).
2. Toda mutación/lectura privada pasa por `getOwned(projectId, ownerId)` → 404 si no es dueño.
3. Assets/export también filtran por `project.ownerId`.
4. Lo público solo si `published && visibility==='public' && status==='active'`.

El campo `role` existe, pero **no** hay bypass admin implementado en ownership.

¿Dónde guardas el token? ¿Cookie o localStorage? ¿Cómo caduca?

Cookie httpOnly `pliego_token` (no localStorage → menos XSS theft).

`SameSite=Lax`, `Secure` en production.

`JWT_EXPIRES_IN` default **7d**; `maxAge` de cookie alineado. Logout limpia cookie.

Frontend usa `credentials: 'include'`.

El texto del usuario acaba en un HTML que renderiza Chromium. ¿Te pueden inyectar algo?

Se escapan `& < > "` en textos, labels, títulos y attrs relevantes (`escapeHtml`). Los `href` se resuelven por whitelist de esquemas (`pliego:` , `http(s)` , `mailto` , `paths` , `page ids`). Riesgo residual: URLs `javascript:` no deberían pasar el resolver; imágenes usan `src` escapado pero el contenido binario ya pasó MIME whitelist. No es un sanitizer HTML completo tipo DOMPurify porque **no se inserta HTML libre del usuario**, solo texto → nodos.

¿Se puede adivinar el enlace público de un proyecto?

Slug = título slugificado + **6 chars base36 aleatorios** (~2.2e9). No es un secreto criptográfico, pero no es secuencial. Solo resoluble si además está publicado. Enumerar a ciegas es costoso; un título muy predecible + fuerza bruta del sufijo es el escenario teórico. Mejora: más entropía o UUIDs opacos.

¿Qué tipos de archivo se pueden subir?

Solo imágenes: **jpeg, png, webp, gif**. Máximo **5 MB**. Se guardan como `{projectId}/{uuid}{ext}` (el nombre del cliente no controla la ruta).

¿Cuántas pruebas tienes y qué cubren?

11 tests Vitest (verificado):

- Backend (9): auth/sesión/proyectos; `resolveInteractionHref` (next/prev/https/page/paths).
- Frontend (2): documento vacío y z-index.

CI en GitHub Actions. **No** hay E2E Playwright amplio ni tests a11y automatizados (roadmap).

¿Está desplegado? ¿Lo puedo abrir ahora?

Sí: **<https://pliego-kr1j.onrender.com>** (health OK en producción). Puede haber cold start en el plan free.

¿Lo han probado usuarios reales?

No hay evidencia en el repositorio (no analytics, no entrevistas versionadas, no issue tracker de usuarios). Para la defensa: di con honestidad si hiciste pruebas con compañeros/tutores y cuántas; no inventes N usuarios si no los documentaste.

¿Cumple WCAG? ¿Se puede usar el editor con teclado o lector de pantalla?

No se reivindica conformidad WCAG AA. Hay bases: `lang="es"`, `:focus-visible`, `aria-label / sr-only / role="alert"` en varias pantallas, `prefers-reduced-motion`. El editor Konva es **débil para teclado/lector** (canvas bitmap-like): la selección/transform es ratón/táctil. Landing/demo son más accesibles que el lienzo.

¿La identidad la has hecho tú? ¿Con qué herramientas?

En código la identidad está **sistematizada** (`pliegoBrand.ts`, CSS variables, SVGs en `frontend/public/brand`, tipografías Google Fonts, plantillas). Herramientas típicas del proyecto: Figma o equivalente para dirección de arte (si aplica en tu proceso), implementación en React/Tailwind, assets SVG.

Afirma solo lo que hayas diseñado tú; el sistema de tokens y la coherencia sí están en el repo.

¿Tanto efecto visual (gradientes, cursor magnético) no molesta en una herramienta de trabajo?

Se separó **marketing/demo** (motion, mesh, beats) del **editor de trabajo** (chrome más quieto, posters de plantilla **estáticos** tras detectar flicker/epileptic motion). Además existe `reduce-motion`. Criterio de producto: la puerta de entrada vende presencia; el lienzo prioriza control. Si molesta, se puede apagar motion sin tocar el modelo de documento.

¿Por qué Space Grotesk y JetBrains Mono?

- **Space Grotesk:** grotesk geométrica contemporánea, legible en UI y titulares cortos; evita Inter/Roboto “AI default”.
- **JetBrains Mono:** metadatos, eyebrows, índices tipográficos (VOL. 03, REPORTAJE) — contraste editorial display vs mono.

Complemento: **Syne** (display brand), **DM Sans** (lectura), **Instrument Serif** (cita editorial).

¿Qué decisión de diseño cambió tu código?

Ejemplos reales del desarrollo:

1. Plantillas con motion infinito → **estáticas** + one-shot fit (parar jitter/epilepsia en móvil).

2. PDF en Render sin `/usr/bin/chromium` → resolver runtime + **@sparticuz/chromium**.
3. Documento como JSON propio (no serializar Konva) → undo/PDF/versión sobre el mismo contrato.
4. Cookie httpOnly en lugar de Bearer en localStorage.
5. Plantillas de 1 página → **3 actos** (portada/cuerpo/cierre) alineados a la narrativa PLIEGO.

¿Qué aporta Pliego frente a Canva?

Canva	PLIEGO
Plantillas masivas, social-first	Editorial digital con dirección de arte propia
Abstrae el documento	JSON transparente, versiones, ownership
Export genérico	PDF con CTAs anotados + publicación <code>/p/:slug</code> con motion
Marca Canva	Marca estudio (tipografía + neón + demo cine)
Collab/enterprise	Slice TFM enfocado, auditable, self-hostable

No compite en catálogo de assets; compite en **control editorial + publicación interactiva + narrativa de producto**.

¿Quién pagaría por esto? ¿Cómo lo sacarías adelante?

- Estudios boutique / marcas culturales / revistas digitales / education design.

Modelo plausible: freemium (1–3 proyectos) + Pro (export PDF, custom domain, assets) + Education.

Siguiente infra: Postgres + S3 + cola PDF + auth durable.

Go-to-market: portfolio cases publicados con PLIEGO, partnerships con escuelas de diseño, open-core demo.

¿Las 400 horas son reales o estimadas?

En el repositorio no hay registro de horas. No digas “400” si no las has medido. Usa: diario de trabajo, commits, fases del README (1–8), y diferencia *estimación de*

planificación vs tiempo real. Si tu memoria del máster pedía 400 h, aclara si es carga ECTS/estimada.

¿Qué ha sido lo más difícil?

Desde el código y la evolución reciente, los focos duros han sido:

1. **Editor móvil:** fit/zoom sin bucles de re-layout.
2. **Fidelidad PDF + Chromium en PaaS free.**
3. **Mantener identidad fuerte sin romper usabilidad del editor.**
4. **Contrato JSON único** para canvas, versiones y export.

Si empezaras de nuevo, ¿qué harías distinto?

- Postgres + storage S3 desde el día 1 del deploy.
- Cola de exportación y límites.
- Migraciones de `document.version` desde el primer cambio de schema.
- E2E Playwright en rutas críticas (login → editar → publish → pdf).
- Conflictos multi-pestaña mínimos (`updatedAt`).
- A11y del editor (lista de capas teclado-first) en paralelo al canvas.

¿Qué has usado de lo aprendido en el máster?

Conecta explícitamente (ajusta a tus asignaturas reales):

- Ingeniería web full stack (API REST, auth, cookies)
- Modelado de datos / Prisma / tradeoff SQL vs documento
- UX/UI y dirección de arte digital
- Arquitectura front (estado, componentes, rendimiento canvas)
- DevOps ligero (Docker, CI, PaaS)
- Metodología de proyecto (fases, roadmap, documentación)

8. Prompt maestro (para pegar a un agente)

Eres ingeniero/a de producto sobre PLIEGO, un estudio editorial visual full stack (React+Vite+Zustand+Konva / Express+Prisma+SQLite / PDF Puppeteer+pdf-lib / publicación /p/:slug / deploy Render). El contrato del documento es JSON version:1 con pages[].elements (text|image|shape|button|video + effects/animation/interaction). Auth: JWT en cookie httpOnly pliego_token; ownership por ownerId. Autosave ~800ms; sin sync multi-tab. Plantillas 3 actos. Identidad: Syne/Space Grotesk/DM Sans/JetBrains Mono; neón #4F80FF, rosa #FF4EDB, ink #050608.

Lee docs/PLIEGO_DOSSIER.md, docs/ARCHITECTURE.md, docs/DATABASE.md, docs/ROADMAP.md y el código antes de proponer cambios. No inventes features (no collab, no pagos, no S3). Prioriza fidelidad del editor, PDF en Render (@sparticuz/chromium), y coherencia de marca.

Tarea: <DESCRIBE AQUÍ LO QUE NECESITAS>

9. Mapa rápido de archivos clave

Tema	Archivo
Brand tokens	<code>frontend/src/brand/pliegoBrand.ts</code>
Store editor / autosave	<code>frontend/src/stores/editorStore.ts</code>
Canvas	<code>frontend/src/components/editor/EditorCanvas.tsx</code>
Plantillas UI	<code>frontend/src/components/dashboard/TemplateCards.tsx</code>
Landing	<code>frontend/src/pages/LandingAuth.tsx</code>
Demo	<code>frontend/src/pages/LiveDemoPage.tsx</code>
Contrato documento	<code>frontend/src/types/document.ts</code> (+ espejo backend)
Schema DB	<code>backend/prisma/schema.prisma</code>
Plantillas server	<code>backend/src/services/templateService.ts</code>
PDF	<code>backend/src/services/exportService.ts</code>
Chromium Resolve	<code>backend/src/utils/chrome.ts</code>
Slug	<code>backend/src/utils/slug.ts</code>
Auth cookie	<code>backend/src/controllers/authController.ts</code>
Ownership	<code>backend/src/services/projectService.ts</code>
Deploy	<code>render.yaml</code> , <code>README.md</code>

10. Frase de cierre para la defensa

PLIEGO no es un clon de Canva: es un estudio editorial en el navegador con un contrato de documento propio, identidad visual coherente, publicación viva y PDF interactivo — un ciclo completo defendible como producto y como ingeniería, con límites conscientes de un TFM (SQLite demo, sin collab, tests aún creciendo).